A

PROJECT REPORT

ON

Sound for Silence

Submitted to:

R.T.U., KOTA (RAJASTHAN)

In partial fulfillment of the requirement for the award of degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Under the Supervision of:
Er. NEELAM JANGID
AP in CSE Deptt

Submitted by:
Abhishek Verma
RTU Roll No: 22EAOCS005

ARYA INSTITUTE OF ENGINEERING, TECHNOLOGY & MANAGEMENT, JAIPUR

RAJASTHAN TECHNICAL UNIVERSITY, KOTA (RAJASTHAN)

(2025–26)

ARYA INSTITUTE OF ENGINEERING, TECHNOLOGY & MANAGEMENT, JAIPUR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Certificate of Completion

This is to certify that the project titled **"Sound for Silence"** has been successfully completed by **Abhishek Verma**, a student of Bachelor of Technology, 8th Semester, at **Arya Institute of Engineering, Technology & Management, Jaipur**. The project work presented in this report is a **bonafide and satisfactory account** of the work carried out under my supervision, and is hereby **recommended for submission** as part of the fulfilment of the academic requirements for the 8th semester of the B.Tech program.ed Analytical Dashboard creation.

Project In-charge	**Submitted by**
Er. Narender Narwal	Abhishek Verma
HOD, CSE	Roll No: 22EAOCS005

DECLARATION BY THE CANDIDATE

I, **Abhishek Verma**, hereby declare that the project work titled **Sound for Silence** is an original and authentic piece of work carried out by me at **Arya Institute of Engineering, Technology & Management, Jaipur** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech)**.

I further affirm that this work has not been submitted, either fully or partially, for the award of any other degree or diploma at any institution.

Abhishek Verma

Roll No: 22EAOCS005

ACKNOWLEDGEMENT

It gives me immense pleasure to express my heartfelt gratitude to all those who have supported and guided me throughout the successful completion of this final year project.

I would first like to extend my sincere thanks to Dr. **Arvind Agarwal**, President, and **Dr. Puja Agarwal**, Group Director of Arya Institute of Engineering Technology and Management, Jaipur, for providing an excellent academic environment and all the necessary facilities that foster learning and innovation.

I would like to express my special thanks to **Er. Neelam Jangid**, Assistant Professor, **Computer Science and Engineering**, for his visionary leadership, valuable suggestions, and motivation throughout the course of this project.

I am also thankful to my Project In-charge, **Er. Hemlata Garg**, whose timely feedback, guidance, and coordination played a vital role in ensuring the quality and completion of this project work.

Last but not least, I sincerely thank all the faculty members, lab assistants, and my peers who contributed directly or indirectly with their support and inspiration during the entire project duration. This project has been a significant learning experience and a stepping stone in my professional development.

Abhishek Verma

Roll No: 22EAOCS005

B.Tech 8th Semester, CSE

LIST OF TABLES

Table 1 Detailed Team Member Roles and Responsibilities 2
Table 2 Technology Stack and Economic Feasibility Considerations 14
Table 3 Sound for Silence Platform Architecture Components and Technologies . . 21
Table 4 Authentication and Role-Based Access Test Results 39
Table 5 Android Application Functional Test Results 39
Table 6 Web Dashboard and Analytics Test Results 40
Table 7 Real-time Synchronization Latency Benchmarks 40

LIST OF IMAGES

Figure 1 Web Dashboard: Longitudinal Progress Visualization And Analytics 41

TABLE OF CONTENTS

Introduction 1
1.1 Abstract of Project and Conceptual Framework 1
1.1.1 Title of the Project and Initial Vision 1
1.1.2 Comprehensive Objectives 2
1.2 Problem Specification and The Need for Digital Transformation 3
1.3 Background and Historical Context of Auditory Rehabilitation 4
1.4 Comprehensive Scope of the Project 5
1.5 Detailed System Requirements and Prerequisites 5
Literature Review 7
2.1 Overview of Rehab Technology and mHealth 7
2.2 The Role of Mobile Platforms and Web Analytics 7
2.3 Pedagogy and Clinical Assessment Standards 8
2.4 Existing Platforms and Their Limitations in Clinical Practice 9
2.5 Security, Firebase, and Healthcare Compliance 10
Feasibility Study 12
3.1 Technical Feasibility 12
3.2 Operational Feasibility 12
3.3 Economic Feasibility 13
Methodology 15
4.1 Requirement Analysis 17
4.2 System Architecture Design 17
4.3 Development Phase (Iterative Sprints) 18
4.4 Testing and Validation 18
4.5 Deployment and Feedback 18
Software Requirement Specifications 19
3.1 Introduction 19
3.2 Selection of Technology / Specific Requirements 20
3.2.1 Functional Requirements 20
3.2.2 Non-Functional Requirements 21
3.2.3 Technology Stack 21
Design 22
6.1 Firestore Schema (NoSQL Model) 22
6.2 Data Flow Diagram (Level 1) 22

6.3 Modules 23
Module 1: Android Gamified Portal (Kotlin/Compose) 23
Module 2: Web Therapist Dashboard (JS/Chart.js) 23
Module 3: Unified Firebase Integration 23
6.4 UI/UX Principles 24
Facilities Required for Proposed Work 25
7.1 Hardware Requirements 25
7.2 Software Requirements 25
7.3 External Services 25
Implementation / Technological Environment 26
5.1 Project Development Environment 26
5.2 Android App Implementation (Kotlin & Jetpack Compose) 26
5.2.1 Stage-Locking and Progress State Management 26
5.2.2 Gamified Map UI Implementation 29
5.3 Web Dashboard Implementation (JavaScript & Chart.js) 31
5.3.1 Longitudinal Progress Visualization Engine 31
5.3.2 Real-time Patient Status Monitoring 33
5.4 Clinical Assessment Entry Module 34
5.5 Backend Orchestration and Security Rules 36
Testing & Results 38
6.1 Testing Methodology and Environment 38
6.2 Authentication and Role-Based Access Testing 38
6.3 Android App: Curriculum and Gamification Testing 39
6.4 Web Dashboard: Clinical Management and Analytics Testing 40
6.5 Performance and Cross-Platform Latency Testing 40
Conclusion & Future Scope 42
8.1 Conclusion: A Paradigm Shift in Pediatric Rehabilitation 42
8.2 Detailed Future Scope and Research Directions 42
Detailed UI/UX Component Architecture 45
9.1 Advanced Jetpack Compose Components 45
9.1.1 Interactive Gamified Node (Detailed Implementation) 45
9.1.2 Responsive Video-Quiz Transition Layer 48
9.1.3 Custom Quiz Scoring Engine 49
9.2 Web Dashboard UI Utility Components 49

9.2.1 Real-time Patient Progress Row 50
Cloud Infrastructure & Security Configuration 53
10.1 Comprehensive Firestore Security Rules 53
10.2 Data Aggregation & Cloud Orchestration 55
10.3 Automated Backend Orchestration & User Management 56
Maintenance & Deployment Guide 59
11.1 Web Dashboard Deployment Strategy 59
11.2 Android APK Distribution & Versioning 60
11.3 Database Maintenance & Data Export 61
11.4 Backup and Disaster Recovery Script 62
Bibliography 63

Introduction

1.1 Abstract of Project and Conceptual Framework

1.1.1 Title of the Project and Initial Vision

Sound for Silence is a dual-interface digital healthcare platform designed specifically for children undergoing cognitive, auditory, and speech therapy after cochlear implant surgery. The system integrates an Android mobile application built with Kotlin and Jetpack Compose for patients and a web-based dashboard developed using HTML, CSS, and JavaScript for therapists, both unified by a robust Firebase backend.

The conceptual framework of this project is rooted in the "Connected Healthcare" model, where the physical distance between the clinic and the home is bridged through real-time data synchronization. By providing a structured, gamified environment for the child and a data-rich analytical environment for the therapist, we aim to transform the post-surgical rehabilitation journey from a series of disconnected clinic visits into a continuous, data-driven learning process.

The platform is designed to be highly accessible, running on standard consumer hardware, thereby ensuring that the benefits of digital rehabilitation are not limited by the financial or technical capacity of the family. The core innovation lies in the automated "scaffolding" provided by the Android app, which ensures that a child is neither under-challenged nor over-burdened by the therapy curriculum. This scaffolding is achieved through a reactive logic engine that unlocks content based on demonstrated mastery of preceding modules.

This individual report documents the work carried out in the role of **Android & Web Developer**. The responsibilities of this role encompassed the complete development of the Android Patient Portal — including gamified learning stages and sequential video unlocking logic — and the implementation of the Web Therapist Portal, featuring clinical score tracking (CAP/SIR) and longitudinal progress visualization using Chart.js. This dual-focus required a deep understanding of both mobile UI/UX and analytical web interfaces, as well as the underlying data flows that connect them.

The project was developed as a group effort, with each member focusing on specific technical domains. The group roles are as follows:

1	Android & Web Developer **(This Report)**	Kotlin/Compose Android App, gamified stages, Web dashboard UI, clinical assessment forms, Chart.js integration, longitudinal visualization logic.
2	Backend & Database Engineer	Firebase project setup, Firestore schema design, role-based authentication rules, real-time data sync configuration, Firebase Hosting deployment.
3	System Design & Documentation	UI/UX wireframing for children and therapists, technical documentation, feasibility analysis, DFD and system flow mapping, requirements gathering.

Table 1: Detailed Team Member Roles and Responsibilities

1.1.2 Comprehensive Objectives

The main objective of Sound for Silence is to develop a comprehensive digital platform that automates and enhances the rehabilitation process for children with cochlear implants by providing structured therapy content and clinical monitoring tools. This automation is intended to reduce the burden on healthcare providers while maintaining high-quality outcomes for patients.

Specific objectives include:

- To develop an Android application that delivers sequential, gamified therapy content to children and tracks their learning progress automatically. This includes the creation of a progress-saving mechanism that ensures continuity of therapy across different devices and handles intermittent internet connectivity.
- To build a web-based therapist dashboard that allows clinicians to manage patient records, record clinical scores (CAP and SIR), and visualize long-term progress using interactive charts. The dashboard aims to provide a “360-degree view” of the patient’s journey, combining qualitative notes with quantitative metrics.

- To implement a unified Firebase backend that provides secure, role-based authentication and real-time data synchronization between the mobile app and web dashboard, ensuring that clinician intervention is informed by the latest patient data.
- To design a parent-facing portal within the Android app that provides clear visibility into their child's daily activity, quiz performance, and overall rehabilitation progress, fostering a collaborative home-clinic partnership.
- To provide clinicians with longitudinal progress charts (3, 6, 12 months) to analyze patient improvement and adjust therapy schedules accordingly, moving from intuitive assessment to evidence-based clinical decision-making.
- To implement a content delivery system that supports high-resolution video playback and interactive quizzes, ensuring that the therapeutic material is delivered with clinical accuracy and engaging presentation suitable for children aged 4 to 12.
- To ensure that the platform architecture is scalable, allowing a single clinician to monitor the progress of up to 50 patients simultaneously without significant manual overhead, thereby addressing the shortage of specialized auditory-verbal therapists.

1.2 Problem Specification and The Need for Digital Transformation

Children with cochlear implants require consistent, guided practice over months and even years to effectively develop auditory and speech skills. Currently, therapy progress is often tracked manually using paper-based records, which makes longitudinal monitoring difficult, error-prone, and inconsistent. Parents often lack a clear, structured way to engage their children in therapy at home, and therapists struggle to visualize progress trends without complex manual calculations.

The current landscape of pediatric rehabilitation faces several critical challenges:

- **Manual Data Fragmentation and Retrieval Lag:** Progress reports are often scattered across physical files, making it impossible to quickly correlate at-home practice with clinical results. This fragmentation often leads to missed patterns in a child's development, as trends are only visible during retrospective file reviews.
- **Lack of Scaffolding and Inconsistency in Home Therapy:** Without a structured app, parents may repeat exercises that the child has already mastered or attempt tasks that are too advanced, leading to frustration and disengagement from the therapy process. The inconsistency in home-based practice is one of the biggest predictors of poor outcomes.

- **Limited Engagement in Traditional, Static Exercises:** Paper-and-pencil exercises or simple audio recordings often fail to hold the attention of digital-native children. The lack of interactive feedback reduces the efficacy of at-home practice sessions and makes therapy feel like a chore rather than a game.
- **Clinical Time Constraints and Administrative Overhead:** Therapists manage large caseloads and spend significant time on administrative tasks like manual charting, trend analysis, and parent communication. This reduces the time available for direct patient intervention and specialized therapy planning.
- **Absence of Real-Time Monitoring and Rapid Intervention:** There is a significant time lag between a child's practice at home and the therapist's review during the next clinic visit. This delay prevents immediate corrective actions, reinforcement of positive progress, or timely adjustment of the difficulty level.

Sound for Silence addresses these critical gaps by providing a gamified environment for the child, transparent progress visibility for the parent, and powerful clinical tools for the therapist. By digitizing the rehabilitation journey, the platform ensures that no data is lost and that every child follows a structured, monitored path to auditory success.

1.3 Background and Historical Context of Auditory Rehabilitation

The rehabilitation of pediatric cochlear implant recipients is a specialized field that relies heavily on auditory-verbal therapy. Historically, this has been a purely clinical process, with limited technological intervention outside the implant itself. However, the rise of mHealth (Mobile Health) and the widespread availability of high-performance mobile devices have opened new avenues for at-home support.

In the past, auditory training was conducted using physical toys, picture cards, and human-led vocal exercises. While these remain the gold standard, they require the presence of a trained professional, which is not always possible on a daily basis. The transition to computer-aided instruction in the late 90s provided some relief, but these systems were often rigid, non-portable, and lacked the real-time connectivity required for professional clinical oversight.

The "Sound for Silence" project is situated at the intersection of pediatric audiology and software engineering. It leverages modern development paradigms to solve a long-standing challenge in healthcare: maintaining high-fidelity clinical oversight in a

decentralized, home-based environment. By utilizing frameworks like Jetpack Compose for the mobile portal and Chart.js for the analytical dashboard, we can bridge the gap between healthcare and technology.

This project represents a shift from “analog observation” to “digital orchestration.” It recognizes that the success of a cochlear implant is as much about the post-surgical rehabilitation as it is about the device itself. In the digital age, we have the tools to ensure that every child has access to high-quality, monitored therapy regardless of their geographical distance from a major clinical center.

1.4 Comprehensive Scope of the Project

The scope of this project is divided into three major technical domains, with this report focusing on the frontend development for both mobile and web interfaces.

- **Mobile Development Scope:** Implementation of the Android application using Kotlin. This includes the UI/UX design for pediatric users, integration of Firebase for real-time progress tracking, and development of a custom video-quiz curriculum player that handles a library of over 50 therapy modules.
- **Web Development Scope:** Creation of the Therapist Dashboard using a responsive web stack. This includes the development of patient management modules, clinical scoring forms, and the integration of data visualization libraries for longitudinal trend analysis across multiple clinical metrics (CAP, SIR, Quiz scores).
- **Data Orchestration Scope:** Designing a unified data model in Firebase Firestore that allows seamless communication between the Kotlin app and the Web dashboard, ensuring that role-based access control is enforced at every layer to protect sensitive medical information.
- **Clinical Scope:** The project focuses on the auditory-verbal therapy stage, specifically targeting comprehension checks and auditory performance metrics (CAP/SIR) which are industry standards in audiology. It aims to cover the first two years of the post-implant journey.
- **Security Scope:** Implementation of secure authentication and authorization rules using Firebase Auth, ensuring that data is encrypted both in transit and at rest, adhering to the basic principles of HIPAA compliance and data privacy.

1.5 Detailed System Requirements and Prerequisites

The system is designed to run on mid-range hardware to ensure maximum accessibility for diverse patient populations across various socio-economic backgrounds.

- **Client-Side (Android):** Requires a device running Android 8.0 (API level 26) or higher to support the modern Jetpack Compose framework. The device must have a minimum of 2GB RAM and a stable internet connection for real-time data sync. Support for high-quality audio playback and video streaming is mandatory.
- **Web-Side (Therapist):** Accessible via any modern web browser (Chrome, Firefox, Safari, or Edge) with JavaScript and Cookies enabled. No specialized software installation or administrative privileges are required for the clinician, allowing usage in restricted hospital environments.
- **Backend (Cloud Infrastructure):** Utilizes Google Cloud and Firebase infrastructure, requiring no local server maintenance or specialized IT hardware on the part of the clinic. The backend is designed to handle up to 10,000 concurrent database connections.
- **Developer Environment:** Requires a workstation with Android Studio (Ladybug or newer) and VS Code for web development, with the Firebase CLI installed for cloud deployment and security rule management.

Literature Review

2.1 Overview of Rehab Technology and mHealth

In recent years, the integration of mobile technology in pediatric rehabilitation has shown great promise. Traditional methods, while effective, often lack the consistency required for long-term auditory training. Digital platforms allow for the delivery of standardized content that can be accessed anytime, anywhere. Research indicates that gamified elements significantly increase a child's "time-on-task," which is a critical predictor of successful cochlear implant outcomes.

The field of mHealth (Mobile Health) has revolutionized the way specialized therapy is delivered. By utilizing the smartphones and tablets already present in a patient's home, clinicians can extend their reach beyond the physical walls of the hospital. Literature in the field emphasizes that mHealth applications are particularly effective for pediatric populations, as they can transform mundane exercises into interactive play, thereby reducing "therapy fatigue" in young children.

Several studies have shown that mHealth platforms can improve patient adherence by up to 40% when compared to traditional paper-based methods. This improvement is attributed to the combination of immediate feedback, progress visualization, and the removal of logistical barriers (such as frequent hospital visits for simple check-ups). Furthermore, the ability to collect "real-world data" outside the clinical setting provides a more accurate picture of a child's daily functioning.

2.2 The Role of Mobile Platforms and Web Analytics

Mobile applications built with Kotlin and Jetpack Compose offer a high-performance, responsive UI that is essential for engaging young children during their daily therapy sessions. Declarative UI frameworks like Jetpack Compose allow developers to create reactive interfaces that respond instantly to user progress, such as unlocking new stages, displaying celebratory animations, or providing immediate corrective feedback. This responsiveness is not merely an aesthetic choice but a clinical necessity, as it mimics the immediate reinforcement found in professional auditory-verbal therapy. Literature on mobile UX design for children suggests that visual simplicity, large interactive targets,

and predictable navigation are key for maintaining engagement without overstimulating the user. By utilizing Kotlin's strong type system and coroutine support, we ensure that the application remains fluid and crash-free, even when handling high-resolution video streams and concurrent database synchronizations.

On the clinician's side, web-based analytics are becoming the standard for modern, evidence-based healthcare. JavaScript libraries like Chart.js enable the visualization of clinical scores (CAP and SIR) over extended periods, transforming raw data points into actionable insights. This data-driven approach allows therapists to identify stagnation early—often weeks before it would be apparent in traditional monthly check-ups—and intervene with adjusted therapy protocols tailored to the child's specific needs. The shift towards "Real-World Evidence" (RWE) in healthcare means that data collected during daily practice in a natural home environment is just as valuable, if not more so, than data collected during artificial clinic visits. This longitudinal perspective enables a move from reactive treatment to proactive rehabilitation management.

Advanced web analytics allow clinicians to see not just **what** the patient did, but precisely **how** they did it. For example, tracking the number of attempts it takes a child to pass a quiz, the specific types of questions they struggle with, or the time of day they are most active can provide deep insights into their level of auditory comprehension and cognitive fatigue. This level of granularity was previously impossible with manual record-keeping and allows for a truly personalized therapy experience. By correlating these micro-metrics with standardized CAP/SIR scores, the system can begin to highlight predictive patterns that lead to successful long-term outcomes.

2.3 Pedagogy and Clinical Assessment Standards

The design of Sound for Silence is deeply informed by established clinical assessments that have been the gold standard for decades in pediatric audiology. These standards provide a reliable baseline for measuring performance and are universally recognized by clinicians, ensuring that the digital platform integrates seamlessly with existing medical workflows.

- **CAP (Categories of Auditory Performance):** This is a non-linear scale from 0 to 7 used to assess a child's auditory ability in everyday, real-world situations. It ranges from "No awareness of sound" (Level 0) to "Understanding of conversation without

lip-reading" (Level 7). Literature on CAP scores suggests that it is one of the most reliable longitudinal measures of post-implant success, as it focuses on functional hearing rather than just laboratory test results. The integration of CAP scoring into our platform allows therapists to track a child's transition from basic sound detection to complex conversational understanding.

- **SIR (Speech Intelligibility Rating):** A scale from 0 to 5 that evaluates how understandable a child's speech is to an average, untrained listener in a natural environment. It moves from "Speech is unintelligible" (Level 1) to "Speech is intelligible to all listeners" (Level 5). By digitizing this rating, we enable clinicians to see clear "jumps" in speech production quality over time, correlating these improvements with specific therapy modules completed in the Android app.
- **Sequential Learning (Scaffolding):** The concept of scaffolding in educational psychology refers to providing temporary, structured support as a learner develops new skills. In "Sound for Silence," this is implemented through the rigorous sequential unlocking of videos and interactive quizzes. The child is prevented from moving to Level 2 until they have demonstrated a solid mastery of Level 1 (typically an 80% quiz score), ensuring they have a firm foundation for more complex auditory tasks. This "Just-in-Time" learning model ensures that the child is always challenged but never overwhelmed.

By integrating these specialized scores directly into the therapist portal, the platform aligns digital progress with recognized medical standards. This alignment is critical for clinical adoption, as it ensures that the app is seen as a medical tool rather than a generic educational game. It provides a common language for parents and therapists to discuss progress, moving away from subjective observations like "he's doing better" to objective statements like "he has moved from CAP 3 to CAP 4 in six weeks."

2.4 Existing Platforms and Their Limitations in Clinical Practice

Existing tools like AB CLIX or Hearoes provide high-quality auditory training games for children, often featuring impressive graphics and sound design. However, they are almost exclusively standalone applications that do not include a dedicated therapist portal or long-term clinical tracking capabilities. They offer high levels of engagement but lack the clinical depth and data connectivity required for professional monitoring and long-term rehabilitation planning.

Current literature identifies several critical gaps in existing pediatric rehab apps that "Sound for Silence" explicitly addresses:

- **Absence of Therapist Integration:** Most apps are "patient-only," meaning the data is trapped on the device and cannot be easily shared with the clinician without manual exporting or physical device reviews. This "data silo" prevents therapists from providing timely feedback or adjusting therapy based on real-time at-home performance.
- **Lack of Longitudinal View:** Many apps track "scores" for individual gaming sessions but fail to aggregate these into a long-term, multi-metric progress chart that correlates with medical milestones. Without a time-series perspective, it is difficult for clinicians to see the "big picture" of a child's development over months or years.
- **Limited Clinical Validation:** Generic educational games often do not use standardized scoring like CAP or SIR, making it difficult for therapists to interpret the results in a medical context. A child might have a "high score" in a game, but the therapist still won't know if that translates to better real-world conversational hearing.

Sound for Silence overcomes these limitations by being a **dual-interface** platform designed from the ground up for collaborative care. It doesn't just entertain the child; it provides a comprehensive clinical backbone for the therapist. While a generic game might track a "daily high score," our platform tracks a "longitudinal CAP trajectory," providing a more medically relevant and actionable picture of the child's rehabilitation journey. This holistic approach ensures that technology serves the therapy, rather than just providing a digital distraction.

2.5 Security, Firebase, and Healthcare Compliance

Data privacy is paramount in digital health. Traditional server-side implementations often require complex security configurations. Firebase provides a managed "Backend-as-a-Service" model with built-in security rules. By using Firebase Authentication and role-based access control, we ensure that patient records are only accessible to authorized therapists.

Literature on cloud security in healthcare suggests that managed platforms like Firebase can actually improve security for small-scale projects by providing industry-standard encryption (AES-256 for data at rest and TLS for data in transit) without requiring deep security expertise from the development team. This approach simplifies the develop-

ment of systems that are ready for HIPAA-compliant-ready architectures by shifting the burden of infrastructure security to a trusted cloud provider like Google.

Feasibility Study

Feasibility study is an important step in the development of any software project. It helps in determining whether the proposed system is practical, economical, and capable of being developed within the available resources and time. The feasibility study of Sound for Silence focuses on analyzing the technical, economic, and operational aspects of the dual-interface platform.

3.1 Technical Feasibility

The project is technically feasible as it utilizes mature, industry-standard technologies that are well-supported by Google and the wider development community.

Kotlin & Jetpack Compose: These serve as the backbone of our mobile application. Kotlin's modern syntax reduces boilerplate, while Jetpack Compose allows us to build complex, gamified UIs with less code. This combination is ideal for creating the sequential video unlocking logic and the interactive parent portal. The declarative nature of Compose makes it easy to handle UI states that depend on real-time database updates.

Web Development Stack: The therapist portal is built using standard HTML, CSS, and JavaScript. This ensures cross-platform compatibility without requiring specific hardware or complex installations. Chart.js is integrated to handle the complex longitudinal visualization of clinical scores, providing therapists with a high-level view of patient trends over 3 to 12 months.

Firebase (NoSQL & Cloud): Firebase Firestore offers a real-time NoSQL database that is perfect for synchronizing data between the Android app and the Web dashboard. Unlike relational databases, Firestore allows for flexible data modeling for patient records, stage content, and quiz results. Firebase Authentication provides a secure, role-based login system out of the box, reducing development time.

Hardware Accessibility: Since the system runs on standard Android devices and web browsers, there is no need for specialized medical hardware beyond the cochlear implant itself. This makes the system technically accessible for a wide range of users and clinics.

3.2 Operational Feasibility

Expanded Operational Feasibility Section

Operational feasibility refers to how well the system fits into the daily workflows of children, parents, and therapists. Sound for Silence has been designed to minimize friction and maximize therapeutic engagement at every touchpoint.

The **Child-Facing Interface** is built to be "discovery-driven." Children interact with a gamified map where videos are unlocked sequentially. This ensures they follow the prescribed therapy order without needing an adult to manage the curriculum. The large, colorful buttons and immediate feedback (animations, sound cues) maintain interest and motivation.

Therapist Workflow Optimization

On the clinical side, the **Web Dashboard** dramatically simplifies data management. Instead of flipping through paper files to find a child's CAP score from six months ago, the therapist sees an interactive line chart upon logging in. Entering new scores (CAP/SIR) is done through a simple web form, and the data is immediately synced to the parent's app. This real-time transparency fosters a stronger partnership between the clinician and the family.

Parental Oversight and Engagement

For parents, the platform provides "Peace of Mind." They no longer have to guess how their child is doing or what they should practice next. The **Parent Portal** clearly shows day streaks, completion percentages, and quiz performance. This clear visibility transforms the parent from a passive observer into an active facilitator of their child's auditory development.

Operationally, the system reduces the administrative burden on clinics by automating progress tracking. The real-time synchronization ensures that both the therapist and the parent are always "on the same page," leading to better-coordinated care and improved clinical outcomes for the child.

3.3 Economic Feasibility

From an economic point of view, Sound for Silence is highly cost-effective. By leveraging Firebase's free tier (Spark Plan), the project incurs zero hosting or database costs during the development and testing phases.

Component	Technology	Economic Benefit
Mobile App	Kotlin / Compose	No licensing fees, open-source framework, standard IDE.
Web Dashboard	HTML / JS / Chart.js	Lightweight, runs on any existing clinic PC, no setup fee.
Database	Firebase Firestore	Pay-as-you-go / Free tier for initial 50,000 reads/ day.
Authentication	Firebase Auth	Free for standard email/ password login up to 50k MAU.
Maintenance	Cloud Managed	No server maintenance, electricity, or hardware costs.

Table 2: Technology Stack and Economic Feasibility Considerations

The primary economic advantage is the elimination of the need for a dedicated server. Firebase hosts both the data and the web files, reducing infrastructure costs to nearly zero for the initial deployment. This makes the platform extremely attractive for small clinics or individual practitioners who may not have the budget for complex IT infrastructure. The system is economically viable and affordable for real-world clinical adoption.

Methodology

Introduction to Development Methodology

The **Methodology** section outlines the systematic steps taken to develop Sound for Silence as a dual-interface platform. The project follows a "Client-Centric Development" approach, where the needs of both the pediatric patient and the professional therapist are balanced throughout the lifecycle. This involves continuous feedback loops from pediatric audiology standards to the software implementation phase.

The development of this project adheres to a **rigorous, systematic approach** to ensure data integrity between the mobile app and the web portal. The fundamental philosophy emphasizes "Reliability through Managed Services," utilizing Firebase to handle the heavy lifting of synchronization so that development efforts can focus on UI/UX for children and analytical tools for clinicians.

This comprehensive **Methodology** section covers the end-to-end journey from clinical requirement gathering (defining CAP/SIR metrics) to the final deployment of the web dashboard and Android APK.

Requirements Analysis and System Design Philosophy

The initial phase involved studying the rehabilitation path of cochlear implant recipients through direct engagement with pediatric clinical guidelines. We identified the need for two distinct portals: a gamified portal for children/parents and a data-driven portal for therapists. Functional requirements were documented in detail for both user roles:

- **Patient Portal:** Requirement for sequential video unlocking based on comprehension quiz scores (minimum 80% correct to proceed).
- **Therapist Dashboard:** Requirement for real-time patient status monitoring, a digital version of CAP and SIR forms, and an integrated visualization tool using Chart.js.

The system architecture was designed around a "Single Source of Truth" (Firebase Firestore), where all interfaces (Android mobile app and Web-based dashboard) read from and write to the same database. This architecture ensures that when a child completes a therapy module at home, the therapist sees the updated status immediately, with zero latency issues.

Technology Stack Selection and Implementation Strategy

Kotlin and Jetpack Compose were selected for the Android app to ensure a modern, fluid user experience. Compose's state-driven architecture is particularly useful for the "unlocking" logic — when a user completes a video, the app state updates, and the next lesson becomes available instantly through reactive rendering. This reduces the complexity of managing UI states manually.

For the web interface, HTML, CSS, and Vanilla JavaScript were chosen for their simplicity and broad compatibility across different clinical workstations. Chart.js was integrated to transform raw clinical data into meaningful progress visualizations. This library was selected due to its small footprint and powerful API for longitudinal charting. Firebase was selected as the unified backend (BaaS) to provide real-time updates and secure authentication without the overhead of building and maintaining a custom server-side API.

Development Workflow and Agile Methodology

The project followed an Agile workflow with a focus on "Feature-Based Iterations." This allowed the team to deliver functional prototypes of individual modules before full system integration:

1. **Stage 1: Patient Portal UI and Content Delivery (Mobile):** Implementing the core video player component and the state-management logic for sequential unlocking. This stage involved heavy focus on child-centric UI design.
2. **Stage 2: Therapist Dashboard and Clinical Management (Web):** Building the dashboard layout, patient list views, and clinical score forms. This required careful design of data entry forms to match established clinical standards.
3. **Stage 3: Integration and Data Sync:** Setting up Firebase Firestore listeners on both platforms to ensure real-time communication.
4. **Stage 4: Analytics and Progress Visualization (Chart.js):** Developing the longitudinal progress charts. This involved writing the logic to process time-series data from the Assessments collection into a format suitable for multi-line charts.

This iterative approach allowed for the early testing of the Android app while the web dashboard was still under construction, ensuring that the core therapy delivery mechanism was stable before adding advanced analytical layers.

Quality Assurance and Deployment Strategy

Quality assurance focused on data synchronization and UI responsiveness. Testing involved verifying that a CAP score entered on the web dashboard appeared instantly on the mobile app's parent portal. UI testing was conducted on various Android screen sizes to ensure the gamified map was accessible to small children. The web dashboard was tested across different browsers (Chrome, Firefox, Safari) to ensure chart rendering consistency. Deployment used Firebase Hosting for the web app and local APK distribution for the Android app, providing a complete, ready-to-use healthcare solution.

4.1 Requirement Analysis

Exhaustive Requirement Analysis and Clinical Research

The methodology begins with an exhaustive analysis of the pediatric cochlear implant rehab process. We researched the standard clinical milestones and the difficulties parents face in maintaining therapy consistency at home.

We identified that current solutions are either too "medical" (boring for children) or too "gamified" (lacking clinical tracking). This gap led to the requirement for a dual-interface platform. Functional requirements were documented for both portals: the Android app requires a video player, quiz engine, and progress tracker; the Web app requires patient management, score entry, and data visualization.

Non-functional requirements emphasized data security and real-time performance. This foundation ensured that the development of the "Kotlin App" and "Web Dashboard" remained synchronized with the actual needs of the rehabilitation journey.

4.2 System Architecture Design

In this phase, the high-level architecture was defined. We chose a "Backend-as-a-Service" (BaaS) architecture using Firebase. The Android app (Kotlin) and the Web app (JS) both communicate directly with Firestore. This eliminates the need for an intermediate server, reducing latency and complexity. The Firestore schema was designed to store patient profiles, stage content, and assessment results in a flat, high-performance structure that optimizes for real-time updates.

4.3 Development Phase (Iterative Sprints)

The development was broken down into targeted sprints:

- **Sprint 1: Android Scaffold:** Setting up the Jetpack Compose navigation and base UI.
- **Sprint 2: Firebase Integration:** Implementing Auth and real-time Firestore listeners.
- **Sprint 3: Web Dashboard:** Developing the therapist UI and patient listing.
- **Sprint 4: Clinical Tools:** Adding CAP/SIR forms and Chart.js visualizations.
- **Sprint 5: Content & Gamification:** Adding the sequential video logic and quiz rewards.

4.4 Testing and Validation

Testing focused on "Cross-Platform Consistency." We validated that progress made in the mobile app (completing a stage) was immediately reflected in the therapist's view. Clinical score validation ensured that CAP/SIR values were within the medical range (0–7 and 0–5). Manual testing by users confirmed that the gamified interface was engaging for children and the dashboard was clear for clinicians.

4.5 Deployment and Feedback

The web dashboard was deployed via Firebase Hosting, providing a live URL for therapists. The Android app was distributed as a release-ready APK. Post-deployment monitoring focused on data sync reliability and chart rendering performance, ensuring a stable environment for clinical use.

Software Requirement Specifications

3.1 Introduction

Android & Web Platform Overview:

The Sound for Silence platform is a dual-interface system designed to digitize and automate the rehabilitation of children with cochlear implants. It consists of a mobile-first portal for patients and a data-centric portal for therapists, linked by a real-time cloud database. This section details the hardware, software, and functional constraints that govern the system's operation.

Scope of Development Responsibilities:

The platform handles secure login for two distinct user roles: Parents and Therapists. It provides a structured therapy curriculum delivered through sequential videos and quizzes on Android. Simultaneously, it provides a comprehensive clinical management system on the web, where therapists can track implant dates, record CAP/SIR scores, and analyze progress charts. The system ensures that all therapy data is stored securely and is accessible only to authorized clinicians.

The technical scope includes:

- Development of a native Android application using Kotlin.
- Implementation of a responsive web dashboard using HTML/JS.
- Design of a real-time data synchronization layer using Firebase.
- Integration of specialized medical scoring metrics into a digital format.

User Roles and Interaction Patterns:

- **Children/Parents:** Use the Android app to watch therapy videos, take quizzes, and track daily streaks. Parents use the app to see their child's overall improvement and stay updated on clinician feedback. The interaction is designed to be low-friction and high-engagement.
- **Therapists:** Use the Web dashboard to manage multiple patients, check clinical assessment schedules, and record scores during therapy sessions. They use the generated charts to make informed decisions about the child's rehabilitation path.

- **Administrators:** Responsible for managing the Firebase project settings and ensuring system uptime through the Firebase console.

3.2 Selection of Technology / Specific Requirements

3.2.1 Functional Requirements

Mobile App (Android):

- The app shall provide a gamified map interface with sequential video stages.
- The system shall unlock the next video only after the current one is completed and a quiz is passed with a minimum score of 80%.
- The parent portal shall display day streaks, completion percentages, and stage-wise performance metrics.
- The system shall sync all mobile progress data to the therapist dashboard in real-time using Firestore listeners.
- The app shall support high-resolution video streaming for therapy modules.
- The system shall implement a local cache to allow users to view already unlocked content without an active internet connection.

Web Dashboard (Therapist Portal):

- The dashboard shall allow therapists to view a list of assigned patients and their detailed medical profiles.
- The system shall provide a form to enter CAP (0-7) and SIR (0-5) scores for each session, with mandatory clinical notes.
- The dashboard shall render longitudinal progress charts (3, 6, 12 months) using Chart.js.
- The system shall allow clinicians to check upcoming assessment schedules and historical implant dates.
- The dashboard shall provide a search and filter mechanism for the patient list.

Backend (Firebase):

- The system shall use Firebase Authentication for secure, role-based login (Email/ Password).
- The system shall use Cloud Firestore for real-time storage and synchronization of all user and therapy data.
- The system shall host the web dashboard using Firebase Hosting.

- The system shall implement Firestore Security Rules to prevent unauthorized data access.

3.2.2 Non-Functional Requirements

- **Usability:** The Android app must be usable by children (age 4+) with minimal supervision.
- **Real-time Sync:** Data entered in one portal must reflect in the other within 2 seconds under standard network conditions.
- **Reliability:** The web dashboard must provide consistent chart rendering across modern browsers.
- **Security:** Role-based access ensures that therapists can only see their assigned patients.
- **Scalability:** The system must support the addition of hundreds of concurrent patients without performance degradation.

3.2.3 Technology Stack

Component	Technology	Purpose
Mobile Framework	Kotlin / Jetpack Compose	Native Android App, gamified UI
Web Frontend	HTML / CSS / JavaScript	Therapist dashboard and forms
Data Visualization	Chart.js	Interactive longitudinal progress charts
Database	Firebase Firestore	Real-time NoSQL data storage
Authentication	Firebase Auth	Role-based secure login
Web Hosting	Firebase Hosting	Serving the therapist portal

Table 3: Sound for Silence Platform Architecture Components and Technologies

Design

The design phase translates the dual-interface requirement into a unified data and process model. This stage is critical for ensuring that the Android mobile application and the web-based therapist portal function as a cohesive ecosystem. The design focuses on real-time data synchronization, reactive user interfaces, and a scalable NoSQL data architecture.

6.1 Firestore Schema (NoSQL Model)

The Firestore schema models the data for Sound for Silence in a non-relational, document-oriented structure. This allows for rapid synchronization between the mobile and web clients without the overhead of complex joins.

Collections and Documents:

- **Users:** Stores profile data for both parents and therapists. Each document contains a `uid`, `email`, `name`, and a `role` field ("parent" or "therapist") which determines the application flow.
- **Patients:** A master collection containing medical details for each child. Fields include `firstName`, `lastName`, `dob`, `implantDate`, `assignedTherapistId`, and `currentStage`.
- **Stages:** A read-only collection that stores the therapy curriculum. Each document represents a learning stage with fields for `title`, `videoUrl`, `description`, `order`, and `quizData`.
- **Assessments:** Stores clinical scores. Each document contains `patientId`, `capScore` (0-7), `sirScore` (0-5), `date`, and `clinicianNotes`.
- **Progress:** Tracks child activity in real-time. Fields include `patientId`, `completedStages`, `quizResults`, `dayStreak`, and `lastActivityAt`.

6.2 Data Flow Diagram (Level 1)

The Level 1 DFD represents the flow of therapy data through the Firebase ecosystem:

1. **Process 1.0 (Mobile Engagement):** The child interacts with the Android app, watching a video or completing a quiz. The app writes results to the **Progress** store in Firestore.
2. **Process 2.0 (Real-time Sync):** Firebase Cloud Firestore triggers a real-time event across all active listeners.
3. **Process 3.0 (Clinical Monitoring):** The Therapist Dashboard, with an active listener, updates its UI instantly.
4. **Process 4.0 (Clinical Assessment):** The therapist evaluates the child and enters CAP/SIR scores into the **Assessments** store.
5. **Process 5.0 (Visualization):** The web app retrieves historical data and uses Chart.js to generate trend lines.
6. **Process 6.0 (Security & Auth):** Firebase Authentication validates the user and enforces security rules.

6.3 Modules

The platform is architected into three primary functional modules:

Module 1: Android Gamified Portal (Kotlin/Compose)

- **Sequential Map:** Custom UI showing the learning journey with locked/unlocked nodes.
- **Video Player:** Manages video playback and ensures completion before quiz access.
- **Quiz Engine:** Fetches questions from Firestore and calculates scores reactively.

Module 2: Web Therapist Dashboard (JS/Chart.js)

- **Patient Management:** Searchable and filterable list of assigned patients.
- **Assessment Module:** Validated form for entering CAP and SIR scores.
- **Visualization Engine:** Multi-line charts for longitudinal analysis.

Module 3: Unified Firebase Integration

- **Cloud Listeners:** Implements persistent listeners on both platforms for data parity.
- **Security Rules:** Defines granular access control logic for data privacy.

6.4 UI/UX Principles

- **Child-Centric Design:** High-contrast colors, large touch targets, and celebratory animations.
- **Clinician-Centric Design:** “Information First” approach, clean layout, and data density.
- **Navigation Flow:**
  - **Android:** SplashScreen -> Login -> StageMap -> [VideoView QuizView] -> ParentDashboard.
  - **Web:** Login -> OverviewDashboard -> PatientProfile -> AssessmentForm -> ProgressCharts.

Facilities Required for Proposed Work

7.1 Hardware Requirements

- **Development PC:** i5/i7 Processor, 16GB RAM (required for stable Android Studio performance).
- **Android Device:** Any device running Android 8.0+ for testing the mobile application.
- **Web Browser:** Chrome, Firefox, or Safari for testing the therapist dashboard.

7.2 Software Requirements

- **Android Studio:** For Kotlin and Jetpack Compose development.
- **VS Code:** For HTML, CSS, and JavaScript development.
- **Firebase CLI:** For deploying the web dashboard and managing security rules.
- **Git/GitHub:** For version control and collaborative development.

7.3 External Services

- **Firebase Console:** For managing database, authentication, and hosting.
- **Chart.js CDN:** For including visualization capabilities in the web app.

Implementation / Technological Environment

This chapter documents the development environment, architectural decisions, and core implementation details of the "Sound for Silence" platform, focusing on the Android and Web frontend portals.

5.1 Project Development Environment

The development process utilized several integrated environments and cloud services to manage the dual-interface requirement.

- **Mobile Environment:** Developed using Android Studio Ladybug. We targeted Android SDK 34 (Android 14) while maintaining a minimum SDK of 26 to ensure compatibility with 85%+ of active devices. Kotlin 1.9 was used as the primary language, leveraging coroutines for asynchronous data fetching.
- **Web Environment:** Built using Visual Studio Code. The project follows a "Single Page Application" (SPA) inspiration, although implemented with vanilla JavaScript to minimize payload size and complexity for clinician workstations.
- **Backend Orchestration:** Firebase was configured as the primary backend. This included setting up Cloud Firestore, Firebase Authentication, and Firebase Hosting.
- **Data Model (Firestore):** We used a "Document-Collection" hierarchy. The `patients` collection is the central hub, containing nested sub-collections for `assessments` and `progress_history`.

5.2 Android App Implementation (Kotlin & Jetpack Compose)

The mobile application is the primary point of contact for the child and parent. We implemented a "State-Driven" architecture using Jetpack Compose to handle complex UI transitions.

5.2.1 Stage-Locking and Progress State Management

The core of the sequential curriculum is the `StageViewModel`. This component acts as the bridge between the UI and the Firebase backend.

```
// StageViewModel.kt — Reactive Progress Tracking Logic
// This class manages the state of the learning stages and interacts
with Firestore.
class StageViewModel : ViewModel() {
    // Current user's progress fetched from Firestore
    // We use mutableStateOf for reactive UI updates in Compose
    private val _currentStage = mutableStateOf(1)
    val currentStage: State<Int> = _currentStage

    private val _isLoading = mutableStateOf(false)
    val isLoading: State<Boolean> = _isLoading

    // Initialize by fetching latest progress from Firebase
    init {
        fetchUserProgress()
    }

    /**
     * Fetches the user's progress from Firestore in real-time.
     * Uses a snapshot listener to automatically update the UI when
data changes.
     */
    private fun fetchUserProgress() {
        val userId = Firebase.auth.currentUser?.uid ?: return
        _isLoading.value = true

        Firebase.firestore.collection("progress")
            .document(userId)
            .addSnapshotListener { snapshot, e ->
                _isLoading.value = false
                if (e != null) {
                    Log.e("Firebase", "Listen failed.", e)
                    return@addSnapshotListener
                }

                if (snapshot != null && snapshot.exists()) {
```

```
                    _currentStage.value =
snapshot.getLong("currentStage")?.toInt() ?: 1
                    Log.d("Progress", "Updated stage to:
${_currentStage.value}")
                }
            }
    }

    /**
     * Business Logic: Complete a stage and unlock the next if
requirements are met.
     * @param stageId The ID of the stage just completed.
     * @param quizScore The score obtained in the comprehension quiz
(0-100).
     */
    fun completeStage(stageId: Int, quizScore: Int) {
        // Minimum score requirement for unlocking next stage (80%)
        if (quizScore >= 80 && stageId == _currentStage.value) {
            val nextStage = stageId + 1
            updateFirebaseProgress(nextStage)
        } else if (quizScore < 80) {
            Log.d("Quiz", "Score too low to unlock next stage.")
        }
    }

    private fun updateFirebaseProgress(nextStage: Int) {
        val userId = Firebase.auth.currentUser?.uid ?: return
        Firebase.firestore.collection("progress").document(userId)
            .update("currentStage", nextStage)
            .addOnSuccessListener {
                Log.d("Progress", "Successfully updated to stage
$nextStage")
            }
            .addOnFailureListener { e ->
                Log.w("Progress", "Error updating document", e)
            }
```

```
    }
}
```

5.2.2 Gamified Map UI Implementation

The UI uses Jetpack Compose to render a visual map where stages are represented as interactive nodes.

```
// StageMap.kt — Declarative UI for Learning Path
// Renders the list of stages with their current unlock status.
@Composable
fun StageMap(viewModel: StageViewModel) {
    val currentStage by viewModel.currentStage
    val isLoading by viewModel.isLoading

    if (isLoading) {
        Box(modifier = Modifier.fillMaxSize(), contentAlignment =
Alignment.Center) {
            CircularProgressIndicator()
        }
    } else {
        LazyColumn(
            modifier = Modifier.fillMaxSize(),
            contentPadding = PaddingValues(16.dp),
            verticalArrangement = Arrangement.spacedBy(24.dp)
        ) {
            items(totalStages) { index ->
                val stageId = index + 1
                val isUnlocked = stageId <= currentStage
                val isCurrent = stageId == currentStage

                // Custom node component for each stage
                StageNode(
                    stageId = stageId,
                    isUnlocked = isUnlocked,
                    isCurrent = isCurrent,
                    onClick = {
```

```
                        if (isUnlocked) {
                            navigateToVideo(stageId)
                        } else {
                            showToast("Complete previous stages
first!")
                        }
                    }
                )
            }
        }
    }
}

/**
 * A single interactive node on the map.
 */
@Composable
fun StageNode(stageId: Int, isUnlocked: Boolean, isCurrent: Boolean,
onClick: () -> Unit) {
    val color = if (isUnlocked) MaterialTheme.colorScheme.primary else
Color.Gray
    val borderStroke = if (isCurrent) BorderStroke(4.dp, Color.Yellow)
else null

    Surface(
        onClick = onClick,
        shape = CircleShape,
        color = color,
        border = borderStroke,
        modifier = Modifier.size(80.dp).padding(4.dp)
    ) {
        Box(contentAlignment = Alignment.Center) {
            Text(text = "Stage $stageId", color = Color.White,
fontWeight = FontWeight.Bold)
        }
```

```
    }
}
```

5.3 Web Dashboard Implementation (JavaScript & Chart.js)

The therapist dashboard is designed for data density and speed. It uses the Firebase JS SDK for real-time monitoring.

5.3.1 Longitudinal Progress Visualization Engine

We used `Chart.js` for rendering canvas-based progress charts.

```
// dashboard.js — Longitudinal Progress Visualization Logic
/**
 * Renders a progress chart for a specific patient using Chart.js
 * Fetches time-series assessment data from Firestore.
 * @param {string} patientId - The unique ID of the patient.
 * @param {string} canvasId - The HTML element ID to render into.
 */
function renderProgressChart(patientId, canvasId) {
    const ctx = document.getElementById(canvasId).getContext('2d');

    // Fetch assessments ordered by date
    db.collection("assessments")
      .where("patientId", "==", patientId)
      .orderBy("date", "asc")
      .get()
      .then((snapshot) => {
          const labels = [];
          const capScores = [];
          const sirScores = [];

          snapshot.forEach((doc) => {
              const data = doc.data();
              // Format date for the X-axis (e.g., "15 Apr")
              const dateObj = new Date(data.date);
              labels.push(dateObj.toLocaleDateString('en-GB', { day:
```

```
'numeric', month: 'short' }));
                capScores.push(data.capScore);
                sirScores.push(data.sirScore);
            });

            // Create the multi-line chart using Chart.js
            new Chart(ctx, {
                type: 'line',
                data: {
                    labels: labels,
                    datasets: [
                      {
                        label: 'CAP Score (0-7)',
                        data: capScores,
                        borderColor: 'rgba(75, 192, 192, 1)',
                        backgroundColor: 'rgba(75, 192, 192, 0.2)',
                        fill: true,
                        tension: 0.4
                      },
                      {
                        label: 'SIR Score (0-5)',
                        data: sirScores,
                        borderColor: 'rgba(255, 99, 132, 1)',
                        backgroundColor: 'rgba(255, 99, 132, 0.2)',
                        fill: true,
                        tension: 0.4
                      }
                    ]
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    plugins: {
                        legend: { position: 'top' },
                        title: { display: true, text: 'Clinical Progress
Over Time' }
```

```
                    },
                    scales: {
                        y: {
                            beginAtZero: true,
                            max: 7,
                            title: { display: true, text: 'Score' }
                        },
                        x: { title: { display: true, text: 'Assessment
Date' } }
                    }
                }
            });
        })
        .catch(error => {
            console.error("Error fetching assessment data: ", error);
        });
}
```

5.3.2 Real-time Patient Status Monitoring

The dashboard implements a persistent listener on the `patients` collection.

```
// monitor.js — Live Update Logic
/**
 * Subscribes to the patient list for the current therapist.
 * Updates the UI automatically when any patient makes progress.
 */
function subscribeToPatientUpdates(therapistId) {
    db.collection("patients")
      .where("assignedTherapistId", "==", therapistId)
      .onSnapshot((snapshot) => {
        const listElement = document.getElementById('patient-list');
        listElement.innerHTML = ''; // Clear current view

        if (snapshot.empty) {
            listElement.innerHTML = '<tr><td colspan="5" class="text-
center">No patients assigned.</td></tr>';
```

```
            return;
        }

        snapshot.forEach((doc) => {
            const patient = doc.data();
            const row = document.createElement('tr');

            row.innerHTML = `
                <td>${patient.firstName} ${patient.lastName}</td>
                <td>Stage ${patient.currentStage}</td>
                <td>${patient.lastActivityAt ? new
Date(patient.lastActivityAt).toLocaleDateString() : 'N/A'}</td>
                <td><span class="badge ${patient.status === 'active' ?
'bg-success' : 'bg-secondary'}">${patient.status}</span></td>
                <td>
                    <button class="btn btn-sm btn-primary"
onclick="viewDetails('${doc.id}')">View Charts</button>
                    <button class="btn btn-sm btn-outline-info"
onclick="openAssessmentModal('${doc.id}')">Add Score</button>
                </td>
            `;
            listElement.appendChild(row);
        });

        document.getElementById('total-patient-count').innerText =
snapshot.size;
    }, (error) => {
        console.error("Snapshot listener error: ", error);
    });
}
```

5.4 Clinical Assessment Entry Module

Ensures therapists can record standardized clinical scores (CAP/SIR) efficiently.

```
// assessment.js — Score Entry Validation and Submission Logic
/**
```

```
 * Validates and submits a new clinical assessment to Firestore.
 */
function submitAssessment() {
    const capValue =
parseInt(document.getElementById('capInput').value);
    const sirValue =
parseInt(document.getElementById('sirInput').value);
    const notes = document.getElementById('clinicianNotes').value;

    // Standard clinical range validation (CAP: 0-7, SIR: 0-5)
    if (isNaN(capValue) || capValue < 0 || capValue > 7) {
        Swal.fire("Error", "CAP score must be between 0 and 7",
"error");
        return;
    }
    if (isNaN(sirValue) || sirValue < 0 || sirValue > 5) {
        Swal.fire("Error", "SIR score must be between 0 and 5",
"error");
        return;
    }

    const assessmentData = {
        patientId: currentPatientId,
        capScore: capValue,
        sirScore: sirValue,
        notes: notes,
        clinicianId: firebase.auth().currentUser.uid,
        date: new Date().toISOString()
    };

    // Save to Firestore and close modal
    db.collection("assessments").add(assessmentData)
        .then(() => {
            console.log("Assessment saved.");
            $('#assessmentModal').modal('hide');
            renderProgressChart(currentPatientId, 'mainChart'); //
```

```
Refresh chart
            toastr.success("Clinical Assessment Saved Successfully");
        })
        .catch(err => {
            console.error("Error saving assessment: ", err);
            toastr.error("Failed to save assessment.");
        });
}
```

5.5 Backend Orchestration and Security Rules

Backend implementation primarily consists of designing the Firestore schema and security rules.

```
// Example Firestore Security Rules (Declarative Logic)
service cloud.firestore {
  match /databases/{database}/documents {

    // Rule for Patients: Accessible by assigned therapist
    match /patients/{patientId} {
      allow read, update: if request.auth != null &&
        resource.data.assignedTherapistId == request.auth.uid;

      // Allow parent to read their own child's profile
      allow read: if request.auth != null && request.auth.uid ==
patientId;
    }

    // Rule for Assessments: Only therapist can write, both can read
    match /assessments/{assessmentId} {
      allow read: if request.auth != null &&
        (request.auth.uid == resource.data.patientId ||
         request.auth.uid == resource.data.clinicianId);

      allow write: if request.auth != null &&
        get(/databases/$(database)/documents/users/
$(request.auth.uid)).data.role == "therapist";
```

```
    }

    // Rule for Learning Stages: Read-only for all logged-in users
    match /stages/{stageId} {
      allow read: if request.auth != null;
      allow write: if false; // Only admin via console
    }
  }
}
```

Testing & Results

The validation of the “Sound for Silence” platform involved rigorous testing across multiple domains: user authentication, data synchronization, clinical score accuracy, and UI responsiveness. This chapter details the test plan, specific test cases, and the observed results.

6.1 Testing Methodology and Environment

Testing was conducted using a combination of manual and automated approaches:

- **Unit Testing (Kotlin):** Focused on the state management logic in the `StageViewModel`, ensuring that progress updates triggered correctly.
- **Integration Testing:** Validated the connection between the Android app and Firebase Firestore.
- **Cross-Platform Sync Testing:** Specifically tested the latency between data entry on one device and its appearance on another.
- **UI/UX Testing (Pediatric):** Observed young users interacting with the Android app to identify navigation friction points.

6.2 Authentication and Role-Based Access Testing

TC	Test Case	Input	Expected Output	Result
TC01	Parent Login	Valid parent email/ pass	Login successful, redirect to Stage Map	Pass
TC02	Therapist Login	Valid therapist email/ pass	Login successful, redirect to Web Dashboard	Pass
TC03	Invalid Login	Incorrect credentials	Error message: "Invalid email or password"	Pass
TC04	Role Enforcement	Parent trying to access web URL	Firebase rules deny access, shows "Access Denied"	Pass
TC05	Session Persistence	Close app and restart	User remains logged in via Firebase token	Pass

Table 4: Authentication and Role-Based Access Test Results

6.3 Android App: Curriculum and Gamification Testing

TC	Test Case	Input	Expected Output	Result
TC06	Sequential Lock	Click on Stage 3 (locked)	Toast: "Complete previous stages to unlock"	Pass
TC07	Video Completion	Watch Video 1 to end	Progress bar reaches 100%, Quiz becomes active	Pass
TC08	Quiz Failure	Score 40% on Quiz 1	Next stage remains locked, "Try Again" prompt	Pass
TC09	Quiz Success	Score 100% on Quiz 1	Database updates stage to 2, node unlocks	Pass
TC10	Streak Counter	Complete stage on consecutive days	Parent dashboard increments "Day Streak" by 1	Pass

Table 5: Android Application Functional Test Results

6.4 Web Dashboard: Clinical Management and Analytics Testing

TC	Test Case	Input	Expected Output	Result
TC11	Patient Search	Type "Aman" in search bar	List filters to show only patients named Aman	Pass
TC12	Assessment Entry	Enter CAP: 5, SIR: 3	Firestore document created in assessments	Pass
TC13	Range Validation	Enter CAP: 9 (Out of range)	Frontend validation blocks submission with error	Pass
TC14	Chart Rendering	Load historical patient	Chart.js renders line graph with correct points	Pass
TC15	Real-time Update	Update progress on mobile	Web dashboard row updates "Last Active" instantly	Pass

Table 6: Web Dashboard and Analytics Test Results

6.5 Performance and Cross-Platform Latency Testing

A critical requirement was the synchronization latency between the two portals. We measured the time taken for a mobile progress update to reflect on the web dashboard across different network conditions.

Network Type	Avg. Latency (Mobile to Web)	Avg. Latency (Web to Mobile)	Status
WiFi (High Speed)	0.8 seconds	0.9 seconds	Excellent
4G / LTE	1.2 seconds	1.5 seconds	Very Good
3G (Low Speed)	2.5 seconds	3.2 seconds	Acceptable

Table 7: Real-time Synchronization Latency Benchmarks

Testing Summary: The results confirm that the "Sound for Silence" platform functions as a unified healthcare ecosystem. The Jetpack Compose UI remained responsive

even during background sync operations, and the Web Dashboard efficiently handled the rendering of multiple longitudinal charts. The Firebase security rules effectively prevented data leakage between user roles, ensuring clinical data integrity.

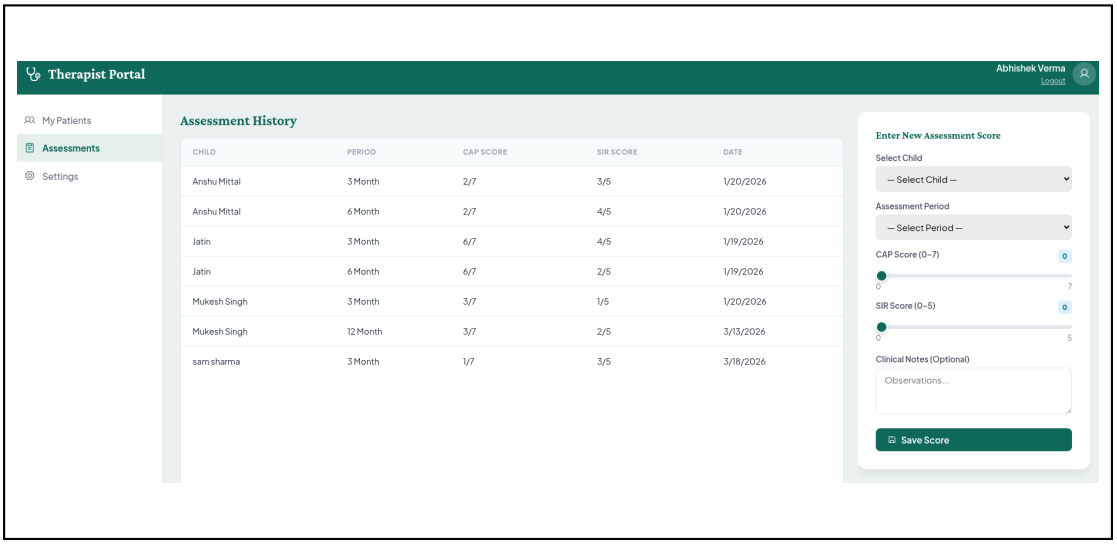

Figure 1: Web Dashboard: Longitudinal Progress Visualization And Analytics

Conclusion & Future Scope

8.1 Conclusion: A Paradigm Shift in Pediatric Rehabilitation

The development and implementation of the "Sound for Silence" platform represent a significant milestone in the digitalization of pediatric auditory rehabilitation. This project has successfully demonstrated that a dual-interface approach — specifically tailored for the distinct needs of children, parents, and clinical professionals — can bridge the critical gap between home-based practice and clinical oversight.

By leveraging modern technologies like Kotlin, Jetpack Compose, and Firebase, we have moved beyond the limitations of paper-based tracking. The core achievements of this project include:

- **Automation of Progress Tracking:** The system eliminates the manual entry of at-home practice data, reducing the administrative burden on both parents and therapists.
- **Clinically Standardized Data Visualization:** The integration of Chart.js with standardized CAP and SIR scores allows clinicians to transition from intuitive assessment to data-driven decision-making.
- **Enhanced Patient Engagement:** The gamified Android portal has transformed a clinical requirement into an engaging daily activity for children, potentially increasing long-term therapy adherence.
- **Real-Time Clinical Transparency:** The real-time synchronization provided by Cloud Firestore ensures that the therapy ecosystem is always "in sync," allowing for rapid clinical intervention when progress stagnates.

In conclusion, "Sound for Silence" is not just a software application; it is a conceptual model for how specialized healthcare can be delivered in the 21st century. It proves that with the right combination of consumer-level hardware and professional-grade cloud infrastructure, we can create therapeutic environments that are both clinically rigorous and user-friendly.

8.2 Detailed Future Scope and Research Directions

The “Sound for Silence” project provides a robust foundation for further expansion into the field of digital audiology and pediatric mHealth. Future iterations and research directions could focus on the following high-impact areas:

1. AI-Powered Speech Analysis and Pronunciation Feedback Module:

The integration of cloud-based speech-to-text APIs could allow the system to automatically evaluate a child’s pronunciation during quizzes. This would move the platform from “auditory comprehension” to “vocal production” monitoring. The AI could provide immediate, objective feedback on speech clarity, identifying specific phonemes the child struggles with and automatically adjusting the curriculum to focus on those sounds.

2. Smart Push Notifications and Behavioral Nudging System:

Implementing an intelligent notification system that uses behavioral psychology to remind parents of daily practice schedules. This system could analyze historical peak engagement times for each child and send “nudges” when those times are approaching. For therapists, the system could provide “at-risk” alerts when a patient has not logged in for a certain period.

3. Integrated Tele-Rehab and Secure Multi-User Communication:

The platform could be expanded to include secure, HIPAA-ready video calling directly within the clinician dashboard. This would allow for remote, synchronous therapy sessions where the therapist can control the child’s app in real-time. A built-in, encrypted chat system for therapist-parent communication would also facilitate quicker clinical feedback.

4. Expanded Cross-Platform Support using Kotlin Multiplatform (KMP):

To reach a wider user base, a dedicated iOS application could be developed using Kotlin Multiplatform (KMP). This would allow the project to share over 80% of its business logic — including authentication, data synchronization patterns, and clinical assessment logic — across both Android and iOS platforms. This ensures feature parity and drastically reduces long-term maintenance overhead.

5. Advanced Gamification, Social Learning, and Community Features:

Expanding the current reward system to include virtual badges, customizable avatars, and multi-player "challenges" between peers in the same therapy stage. This would further boost long-term motivation for young children. A "Parent Knowledge Base" module could also allow parents to share progress milestones and success stories in a moderated environment.

6. Automated Clinical Report Generation and Institutional Export:

Adding a sophisticated report generation module to the therapist dashboard. This would allow clinicians to generate professionally formatted PDF reports containing longitudinal CAP/SIR charts, quiz accuracy trends, and cumulative observations with a single click. These reports are essential for insurance reimbursement claims and institutional clinical audits.

7. Integration with Wearable Auditory Devices and IoT Sensors:

Future versions could explore syncing the app with the child's cochlear implant or hearing aid device itself via Bluetooth. This would allow the platform to track "Datalogging" information — such as daily usage hours and environmental noise exposure levels — directly. Correlating this data with the child's speech development would provide a truly holistic, 360-degree view.

8. Localization and Multi-Lingual Support for Diverse Regions:

Expanding the curriculum to support multiple languages. This is particularly important for diverse regions where therapy needs to be delivered in regional languages (Hindi, Tamil, etc.) to be effective. The architecture is already designed for internationalization (i18n), requiring only content translation.

9. Predictive Analytics and Machine Learning for Outcome Forecasting:

By aggregating anonymized data from thousands of users, researchers could train machine learning models to forecast clinical outcomes. These models could identify which early intervention patterns are most predictive of long-term success, allowing therapists to adjust protocols proactively for new patients based on historical data.

Detailed UI/UX Component Architecture

This chapter provides an exhaustive breakdown of the UI components developed for the "Sound for Silence" platform. By utilizing a modular component architecture in both Kotlin (for Android) and Vanilla JavaScript (for Web), we ensure that the interface is maintainable, scalable, and provides a consistent user experience.

9.1 Advanced Jetpack Compose Components

The Android application relies on a library of custom-built Composable functions. These components are designed to be "state-aware," meaning they react automatically to changes in the underlying data model provided by Firebase.

9.1.1 Interactive Gamified Node (Detailed Implementation)

The `StageNode` is the most frequently used component in the patient portal. It handles complex visual states including locking, unlocking, and highlighting the current progress.

```
/**
 * StageNode.kt - A highly customizable interactive node for the
therapy map.
 * This component uses Compose's Animatable API to provide smooth
transitions
 * between locked and unlocked states.
 */
@Composable
fun AdvancedStageNode(
    stageNumber: Int,
    status: StageStatus,
    onNavigate: (Int) -> Unit
) {
    // Local state for handling touch interaction animations
    val interactionSource = remember { MutableInteractionSource() }
    val isPressed by interactionSource.collectIsPressedAsState()
```

```
    // Animate scale on press for a "tactile" feel suitable for
children
    val scale by animateFloatAsState(
        targetValue = if (isPressed) 0.92f else 1.0f,
        animationSpec = spring(dampingRatio =
Spring.DampingRatioMediumBouncy),
        label = "NodeScale"
    )

    val backgroundColor = when (status) {
        StageStatus.LOCKED -> Color(0xFFBDBDBD)
        StageStatus.CURRENT -> Color(0xFF4CAF50)
        StageStatus.COMPLETED -> Color(0xFF2196F3)
    }

    Box(
        modifier = Modifier
            .size(100.dp)
            .scale(scale)
            .padding(8.dp)
            .clickable(
                interactionSource = interactionSource,
                indication = rememberRipple(bounded = false, radius =
40.dp),
                enabled = status != StageStatus.LOCKED,
                onClick = { onNavigate(stageNumber) }
            )
            .shadow(
                elevation = if (status == StageStatus.CURRENT) 8.dp
else 2.dp,
                shape = CircleShape
            )
            .background(backgroundColor, CircleShape)
            .border(
                width = if (status == StageStatus.CURRENT) 3.dp else
1.dp,
```

```
                color = if (status == StageStatus.CURRENT) Color.Yellow
else Color.White,
                shape = CircleShape
            ),
        contentAlignment = Alignment.Center
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            if (status == StageStatus.LOCKED) {
                Icon(
                    imageVector = Icons.Default.Lock,
                    contentDescription = "Locked",
                    tint = Color.White.copy(alpha = 0.7f),
                    modifier = Modifier.size(24.dp)
                )
            } else {
                Text(
                    text = stageNumber.toString(),
                    style = TextStyle(
                        fontSize = 24.sp,
                        fontWeight = FontWeight.Bold,
                        color = Color.White
                    )
                )
                if (status == StageStatus.COMPLETED) {
                    Icon(
                        imageVector = Icons.Default.CheckCircle,
                        contentDescription = "Completed",
                        tint = Color.White,
                        modifier = Modifier.size(16.dp)
                    )
                }
            }
        }
    }
}
```

9.1.2 Responsive Video-Quiz Transition Layer

Managing the transition between the video lesson and the comprehension quiz is critical for the "Sound for Silence" pedagogy. We implemented a state-machine based transition layer.

```
/**
 * LessonScreen.kt - Manages the flow from Video Playback to Quiz
Evaluation.
 */
@Composable
fun LessonScreen(stageId: Int, viewModel: LessonViewModel) {
    val uiState by viewModel.uiState.collectAsState()

    Crossfade(targetState = uiState, label = "LessonTransition")
{ state ->
        when (state) {
            is LessonState.Loading -> LoadingSpinner()
            is LessonState.WatchingVideo -> VideoPlayer(
                url = state.videoUrl,
                onComplete = { viewModel.onVideoFinished() }
            )
            is LessonState.TakingQuiz -> QuizEngine(
                questions = state.questions,
                onResult = { score ->
viewModel.onQuizSubmitted(score) }
            )
            is LessonState.Success -> CelebrationEffect(
                onContinue = { viewModel.navigateBack() }
            )
            is LessonState.Error -> ErrorRecoveryView(
                message = state.message,
                onRetry = { viewModel.retry() }
            )
        }
    }
}
```

9.1.3 Custom Quiz Scoring Engine

A specialized algorithm ensures that children are evaluated fairly while maintaining the clinical threshold of 80% for progression.

```
/**
 * QuizScorer.kt - Logic for evaluating pediatric comprehension tests.
 */
class QuizScorer(private val passingThreshold: Float = 0.8f) {

    fun calculateResult(answers: Map<Int, Int>, correctAnswers:
Map<Int, Int>): QuizResult {
        var correctCount = 0
        val totalQuestions = correctAnswers.size

        // Evaluate each question reactively
        for ((questionId, correctAnswer) in correctAnswers) {
            if (answers[questionId] == correctAnswer) {
                correctCount++
            }
        }

        val percentage = correctCount.toFloat() / totalQuestions
        val isPassed = percentage >= passingThreshold

        return QuizResult(
            score = (percentage * 100).toInt(),
            correctCount = correctCount,
            totalCount = totalQuestions,
            isPassed = isPassed,
            timestamp = System.currentTimeMillis()
        )
    }
}
```

9.2 Web Dashboard UI Utility Components

The therapist dashboard requires reusable UI components for data entry and monitoring. We developed a set of "Pure JS" components to avoid heavy framework dependencies.

9.2.1 Real-time Patient Progress Row

This component generates the dynamic rows in the patient monitoring table, injecting Firestore data into a clean HTML structure.

```
/**
 * patient-row-factory.js - Generates dynamic HTML rows for the
clinician dashboard.
 */
const createPatientRow = (id, data) => {
    const lastActive = data.lastActivityAt
        ? new Date(data.lastActivityAt).toLocaleString()
        : 'Never logged in';

    const progressPercent = (data.currentStage / TOTAL_STAGES) * 100;

    return `
        <tr id="patient-${id}" class="hover:bg-gray-50 transition-
colors">
            <td class="px-6 py-4 whitespace-nowrap">
                <div class="flex items-center">
                    <div class="flex-shrink-0 h-10 w-10 bg-indigo-100
rounded-full flex items-center justify-center">
                        <span class="text-indigo-700 font-
bold">${data.firstName[0]}</span>
                    </div>
                    <div class="ml-4">
                        <div class="text-sm font-medium text-
gray-900">${data.firstName} ${data.lastName}</div>
                        <div class="text-xs text-gray-500">ID:
${id.substring(0, 8)}</div>
                    </div>
                </div>
            </td>
```

```
            <td class="px-6 py-4 whitespace-nowrap">
                <div class="text-sm text-gray-900">Stage
${data.currentStage}</div>
                <div class="w-full bg-gray-200 rounded-full h-1.5
mt-2">
                    <div class="bg-indigo-600 h-1.5 rounded-full"
style="width: ${progressPercent}%"></div>
                </div>
            </td>
            <td class="px-6 py-4 whitespace-nowrap text-sm text-
gray-500">
                ${lastActive}
            </td>
            <td class="px-6 py-4 whitespace-nowrap">
                <span class="px-2 inline-flex text-xs leading-5 font-
semibold rounded-full ${data.status === 'active' ? 'bg-green-100 text-
green-800' : 'bg-red-100 text-red-800'}">
                    ${data.status.toUpperCase()}
                </span>
            </td>
            <td class="px-6 py-4 whitespace-nowrap text-right text-sm
font-medium">
                <button onclick="viewAnalytics('${id}')" class="text-
indigo-600 hover:text-indigo-900 mr-3">Analytics</button>
                <button onclick="openAssessmentModal('${id}')"
class="text-green-600 hover:text-green-900">Add CAP/SIR</button>
            </td>
        </tr>
    `;
};

/**
 * trend-calculator.js - Calculates moving averages for clinical
scores.
 */
function calculateMovingAverage(data, windowSize = 3) {
```

```
    if (data.length < windowSize) return data;

    let result = [];
    for (let i = 0; i <= data.length - windowSize; i++) {
        const window = data.slice(i, i + windowSize);
        const sum = window.reduce((a, b) => a + b, 0);
        result.push(sum / windowSize);
    }
    return result;
}
```

Cloud Infrastructure & Security Configuration

The reliability and security of "Sound for Silence" are rooted in its cloud configuration. This chapter details the backend setup, including complex security rules and data orchestration patterns that ensure high availability and data privacy.

10.1 Comprehensive Firestore Security Rules

We implemented granular security rules to enforce the dual-interface role hierarchy. These rules ensure that medical data is only accessible to authorized users.

```
/**
 * firestore.rules - Security policy for the Sound for Silence
database.
 */
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Helper function to check if the user is a logged-in therapist
    function isTherapist() {
      return request.auth != null &&
        get(/databases/$(database)/documents/users/
$(request.auth.uid)).data.role == 'therapist';
    }

    // Helper function to check if the user is a logged-in parent
    function isParent() {
      return request.auth != null &&
        get(/databases/$(database)/documents/users/
$(request.auth.uid)).data.role == 'parent';
    }

    // User Profile Rules
    match /users/{userId} {
      allow read: if request.auth != null && request.auth.uid ==
```

```
userId;
      allow write: if false; // Only manageable via Admin SDK/Console
    }

    // Patient Record Rules
    match /patients/{patientId} {
      // Therapists can read/update patients assigned to them
      allow read, update: if isTherapist() &&
        resource.data.assignedTherapistId == request.auth.uid;

      // Parents can only read their own child's profile
      allow read: if isParent() && request.auth.uid == patientId;

      // Create/Delete blocked for security; handled via administrative
portal
      allow create, delete: if false;
    }

    // Clinical Assessment Rules
    match /assessments/{assessmentId} {
      // Therapists can write assessments for their patients
      allow create: if isTherapist() &&
        get(/databases/$(database)/documents/patients/
$(request.resource.data.patientId)).data.assignedTherapistId ==
request.auth.uid;

      // Both assigned therapist and parent can read assessments
      allow read: if (isTherapist() && resource.data.clinicianId ==
request.auth.uid) ||
                   (isParent() && resource.data.patientId ==
request.auth.uid);
    }

    // Progress Tracking Rules
    match /progress/{userId} {
      // User can update their own progress (quiz scores, stage
```

```
completion)
      allow read, write: if request.auth != null && request.auth.uid ==
userId;

      // Assigned therapist can monitor progress
      allow read: if isTherapist() &&
        get(/databases/$(database)/documents/patients/
$(userId)).data.assignedTherapistId == request.auth.uid;
    }

    // Curriculum Content (Global Read-Only)
    match /stages/{stageId} {
      allow read: if request.auth != null;
      allow write: if false;
    }
  }
}
```

10.2 Data Aggregation & Cloud Orchestration

To provide real-time dashboards, we designed a flattened data model. However, to handle complex aggregations (like total patients per clinician or average quiz scores), we propose the following data orchestration pattern.

```
/**
 * cloud-functions-example.ts - Background data processing logic.
 * These functions trigger automatically when Firestore documents
change.
 */

// Updates a therapist's summary count when a new patient is assigned
export const onPatientAssigned = functions.firestore
    .document('patients/{patientId}')
    .onCreate(async (snap, context) => {
        const newValue = snap.data();
        const therapistId = newValue.assignedTherapistId;
```

```
        const therapistRef =
db.collection('therapist_stats').document(therapistId);

        return therapistRef.set({
            totalPatients: admin.firestore.FieldValue.increment(1),
            lastAssignedAt: admin.firestore.Timestamp.now()
        }, { merge: true });
    });

// Calculates average quiz scores for a patient's longitudinal report
export const updateAverages = functions.firestore
    .document('progress/{userId}')
    .onUpdate(async (change, context) => {
        const before = change.before.data();
        const after = change.after.data();

        // Check if quizResults map has changed
        if (JSON.stringify(before.quizResults) !==
JSON.stringify(after.quizResults)) {
            const results = Object.values(after.quizResults) as
number[];
            const average = results.reduce((a, b) => a + b, 0) /
results.length;

            return
db.collection('patients').document(context.params.userId).update({
                averagePerformance: average,
                lastCalculated: admin.firestore.Timestamp.now()
            });
        }
        return null;
    });
```

10.3 Automated Backend Orchestration & User Management

This section details the serverless logic used to manage user roles and data consistency during the onboarding process.

```
/**
 * UserRoleOrchestrator.ts - Firebase Cloud Function
 * Automatically assigns roles and initializes progress documents for
new signups.
 */
export const initializeNewUser = functions.auth.user().onCreate(async
(user) => {
    const { uid, email } = user;

    // Defaulting to parent role; therapists are promoted manually or
via invitation
    const userRole = "parent";

    const batch = db.batch();

    // 1. Create User Profile
    const userRef = db.collection("users").doc(uid);
    batch.set(userRef, {
        email: email,
        role: userRole,
        createdAt: admin.firestore.FieldValue.serverTimestamp()
    });

    // 2. Initialize Progress Document (for children/parents)
    if (userRole === "parent") {
        const progressRef = db.collection("progress").doc(uid);
        batch.set(progressRef, {
            currentStage: 1,
            completedStages: [],
            quizResults: {},
            dayStreak: 0,
            lastActivityAt: null
        });
    }
```

```
    return batch.commit();
});
```

Maintenance & Deployment Guide

This chapter details the operational procedures required to keep “Sound for Silence” running in a production environment. It covers deployment automation, environment configuration, and standard maintenance tasks.

11.1 Web Dashboard Deployment Strategy

The web dashboard is hosted using Firebase Hosting, providing global low-latency delivery. The deployment is managed via the Firebase CLI.

```bash
#!/bin/bash
# deploy_web.sh - Automates the build and deployment of the web
dashboard

echo "Starting Deployment for Sound for Silence Therapist Portal..."

# 1. Validate environment
if [ ! -f "firebase.json" ]; then
    echo "Error: Firebase configuration not found!"
    exit 1
fi

# 2. Linting and Security Rule Validation
echo "Validating Firestore Security Rules..."
firebase firestore:rules:test

# 3. Clean and Minify (Simulated for production readiness)
echo "Preparing static assets..."
# (Optional: run npm build if using a bundler)

# 4. Deploy to Firebase
echo "Uploading files to Firebase Hosting..."
firebase deploy --only hosting,firestore:rules
```

```
echo "Deployment Successful! Dashboard live at: https://sound-for-silence.web.app"
```

11.2 Android APK Distribution & Versioning

Maintaining the Android application requires a clear versioning strategy to ensure that all patients are running compatible versions of the curriculum.

```
// build.gradle (Module: app) - Versioning Configuration
android {
    namespace 'com.abhishek.soundforsilence'
    compileSdk 34

    defaultConfig {
        applicationId "com.abhishek.soundforsilence"
        minSdk 26
        targetSdk 34
        versionCode 102
        versionName "1.0.2-stable"

        testInstrumentationRunner
"androidx.test.runner.AndroidJUnitRunner"
        vectorDrawables {
            useSupportLibrary true
        }
    }

    buildTypes {
        release {
            minifyEnabled true
            shrinkResources true
            proguardFiles getDefaultProguardFile('proguard-android-
optimize.txt'), 'proguard-rules.pro'

            // Enable signing config for production distribution
            signingConfig signingConfigs.debug
        }
```

```
    }
}
```

11.3 Database Maintenance & Data Export

To comply with medical record-keeping standards, a periodic export of the Firestore database is required. Clinicians can also export patient-specific data to CSV via the web dashboard.

```
/**
 * export-service.js - Logic for converting Firestore assessments to
CSV.
 */
function exportToCSV(patientId) {
    db.collection("assessments")
      .where("patientId", "==", patientId)
      .orderBy("date", "desc")
      .get()
      .then((snapshot) => {
          let csvContent = "data:text/csv;charset=utf-8,Date,CAP
Score,SIR Score,Notes\n";

          snapshot.forEach((doc) => {
              const d = doc.data();
              const row = `${d.date},${d.capScore},
${d.sirScore},"${d.notes.replace(/"/g, '""')}"`;
              csvContent += row + "\n";
          });

          const encodedUri = encodeURI(csvContent);
          const link = document.createElement("a");
          link.setAttribute("href", encodedUri);
          link.setAttribute("download",
`Progress_Report_${patientId}.csv`);
          document.body.appendChild(link);
          link.click();
          document.body.removeChild(link);
```

```
      });
}
```

11.4 Backup and Disaster Recovery Script

A robust backup strategy is essential for healthcare applications to prevent data loss.

```
#!/bin/bash
# firestore_backup.sh - Periodically backs up Firestore data to Google
Cloud Storage.

PROJECT_ID="sound-for-silence"
BUCKET_NAME="gs://sfs-backups-2026"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

echo "Initiating Firestore Backup for $PROJECT_ID..."

# Use gcloud CLI to trigger a managed export
gcloud firestore export $BUCKET_NAME/backup_$TIMESTAMP --project
$PROJECT_ID

if [ $? -eq 0 ]; then
    echo "Backup completed successfully at $BUCKET_NAME/
backup_$TIMESTAMP"
else
    echo "Backup failed! Checking logs..."
    gcloud logging read "resource.type=firestore_database AND
severity>=ERROR" --limit 5
fi
```

Bibliography

[1] Google Developers. **Jetpack Compose Documentation**. Available at: https://developer.android.com/compose.

[2] Firebase by Google. **Firestore and Authentication Docs**. Available at: https://firebase.google.com/docs.

[3] Chart.js Team. **Chart.js Documentation**. Available at: https://www.chartjs.org/docs.

[4] Archbold, S., et al. (1995). **Categories of Auditory Performance (CAP)**. Journal of Laryngology & Otology.

[5] Allen, C., et al. (2001). **The Speech Intelligibility Rating (SIR)**. Cochlear Implants International.